+ New

I have some homework coding tasks. I'll give you instructions and requirements. Please help me to fill the codes.

Absolutely, Xi Cheng! I'd be happy to help you with your coding homework. Please provide the instructions and requirements for your task—such as the programming language, what the code should do, any specific functions or logic you need to implement, and any constraints or examples you have.

You can also share any code you've already written, and I'll help you fill in the gaps or debug it. Let's get started!

1:04pm

Gradient Descent with Momentum
Implement the gradient descent with momentum algorithm. **Fill in the missing code** for updating the parameters. As a verification step, compare the final loss with the previous part to ensure it is reasonable and not significantly different.

```
def runGDM(maxiter, stepsize, beta):
    w = np.zeros((2, 1))
    grads_m = []
    ws_m = []
    losses_m = []
    for i in range(maxiter):
        grad = 2 * (X.T @ X @ w) - 2 * X.T @ y
        if i == 0:
            smoothed_grad = grad

    ################################################################################
    #
        # TODO: implement this method

    ################################################################################
    #

    ################################################################################
    #
        w = w - stepsize * smoothed_grad
        grads_m.append(grad)
        ws_m.append(w)
        loss = np.linalg.norm(y - X @ w) ** 2
        losses_m.append(loss)
    print("Final loss =", loss)
    grads_m = np.array(grads_m).squeeze()
    ws_m = np.array(ws_m).squeeze()
    return grads_m, ws_m, losses_m
maxiter = 100
stepsize = 1e-4
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize, beta)
```

Fill in ONLY the TODO
Keep all other code unchanged

Here I copied and pasted the tasks and asked the Mistral to fill th TODO. Also, I emphasized that it should not change any other part except the TODO.

To implement **gradient descent with momentum**, you need to update the `smoothed_grad` variable using the momentum term. The momentum update rule is:

$$v_t = \beta v_{t-1} + (1 - \beta)\nabla f(w_t)$$

$$w_{t+1} = w_t - \text{stepsize} \cdot v_t$$

Where:

- v_t is the "velocity" (or smoothed gradient),
- β is the momentum parameter,
- $\nabla f(w_t)$ is the current gradient.

Fill in the TODO section as follows:

Python Copy

```python
smoothed_grad = beta * smoothed_grad + (1 - beta) * grad
```

Mistral correctly provided the code answer with the explanation.

This line updates the smoothed gradient using the momentum term. The rest of your code remains unchanged.

Great! Let's move to the next: Pleas ONLY fill the TODO

For the next part, I uploaded some pre-intructions and the main question then ask the Mistral to fill the TODO.

To re-run one of the methods (e.g., gradient descent) with a different learning rate, you can modify the `stepsize` parameter in the function call. Here's how you can implement the TODO section to compare the results with a different learning rate:

Python Copy

```python
# TODO: implement this method
maxiter = 100
stepsize = 1e-3  # Changed learning rate
beta = 0.6
grads, ws, losses = runGD(maxiter, stepsize)  # Assuming runGD is your gradient descent function
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize, beta)
```

This code snippet changes the learning rate to `1e-3` and re-runs the gradient descent method. You can then compare the results (`grads`, `ws`, and `losses`) with those from the gradient descent with momentum method (`grads_m`, `ws_m`, and `losses_m`).

The question asks you to further accelerate the method, so you should only change the learning rate of the faster one

Mistral gave an answer which both changed the learning rate of GD and GDM. However, the assignment asked to change only the faster one to achieve further acceleration. So I gave the hint as a follow-up. Then, the Mistral got the correct answer.

To further accelerate the convergence of the faster method (gradient descent with momentum), you should change the learning rate (`stepsize`) in the `runGDM` function call. Here's how you can do it:

Python Copy

```python
# TODO: implement this method
maxiter = 100
stepsize_momentum = 1e-3  # New learning rate for momentum
beta = 0.6
grads_m_fast, ws_m_fast, losses_m_fast = runGDM(maxiter, stepsize_momentum, beta)
```

This code snippet changes the learning rate for the gradient descent with momentum method to `1e-3`. You can then compare the results (`grads_m_fast`, `ws_m_fast`, and `losses_m_fast`) with the original gradient descent method.

2:02pm

Ask Le Chat anything

Research Think Tools

Le Chat can make mistakes. Check important information.